14 ГБ -> 0,46 ГБ

Обучение Qwen2.5-1.5B (NF4 + LoRA) на Kaggle T4 на своих Triton-кернелах

Команда: [название команды]

1,860x

быстрее вся команда soup train
(79,0 -> 42,5 с)

30,98x

меньше пик памяти GPU
(14 345 -> 463 MiB)

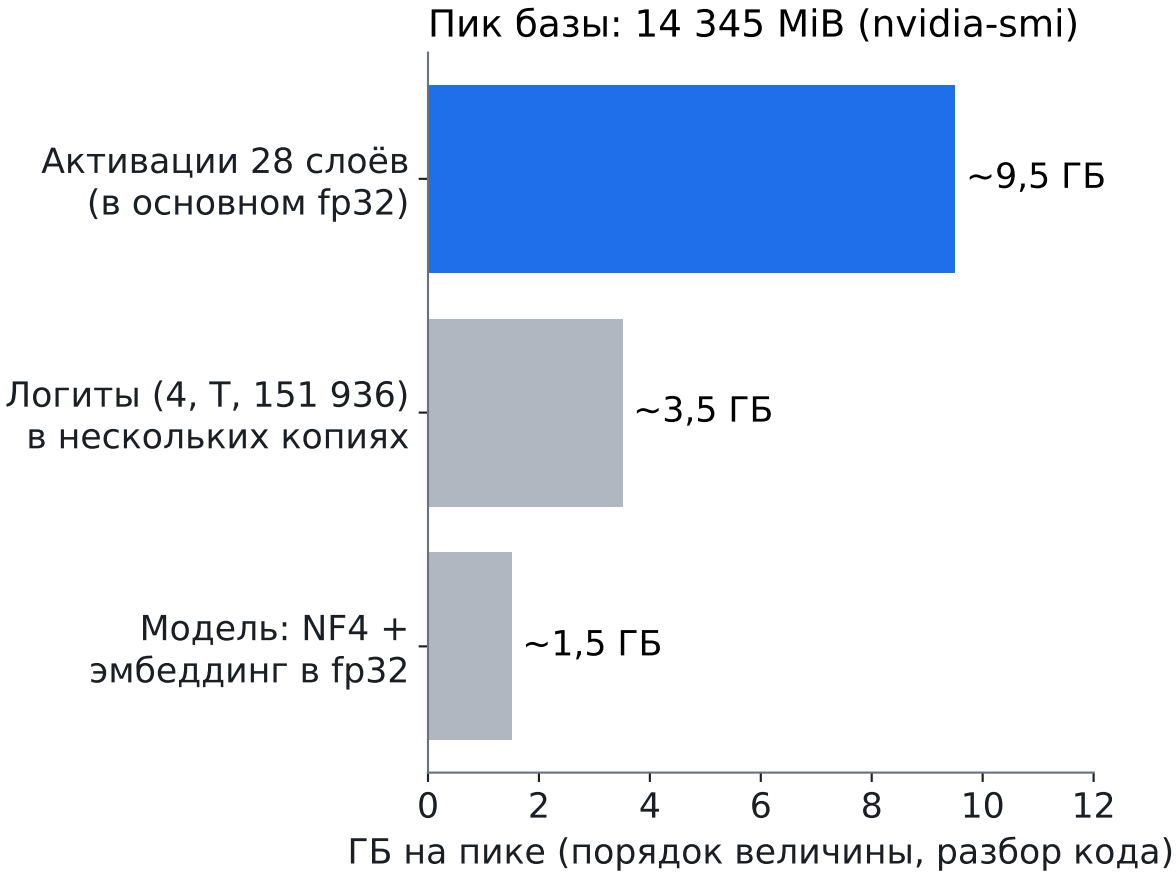
10 / 10

проверок check_rules.py,
eval loss -0,003%

Тот же адаптер на тех же батчах: модель, данные, LoRA, оптимизатор и выходы Soup не меняются. Kaggle T4, --repeats 2, код 838827f.

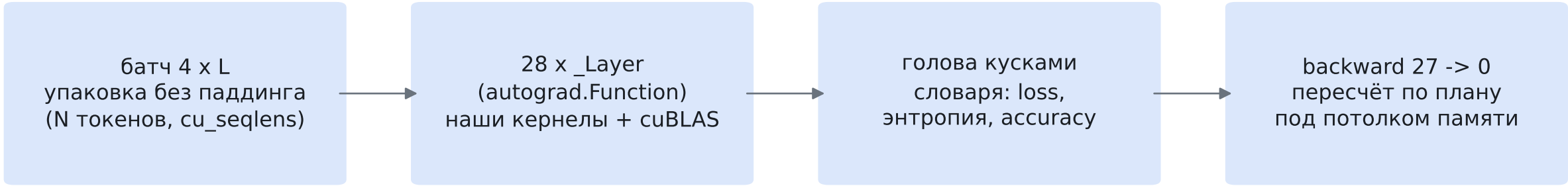
Задача и база: куда уходят 14 ГБ

- База: soup train 79,0 с, из них цикл 64 шагов 57,5 с; пик 14 345 из 15 ГБ.
- bitsandbytes возвращает выход каждой проекции в fp32: gate/up MLP (токены x 8 960) хранятся для backward в fp32.
- peft поднимает эмбеддинг / lm_head (151 936 x 1 536) в fp32.
- Паддинг: ~205 реальных токенов на строку при длине батча ~380.
- Логиты на весь словарь: fp16, fp32 для loss, градиент, копия для accuracy.



Память съедают не веса, а то, что модули HF хранят между собой: менять надо весь шаг.

Идея: заменить весь шаг, сохранив математику базы



CPU -> GPU, по слою вперёд: NF4-веса слоя (2 слота по 23 MiB)

GPU -> CPU -> GPU: входы слоёв, выходы [gate | up] и (от 9 ГБ/с) q, k/v, attention, o_proj

CPU -> GPU: куски lm_head по 8 192 строки (3 буфера по 24 MiB, 2 - ещё во forward)

GEMM - cuBLAS, всё между ними - наши кернелы. На T4 tl.dot в Triton 3.6 без тензорных ядер: ~1 TFLOPS против 16-34 у cuBLAS.

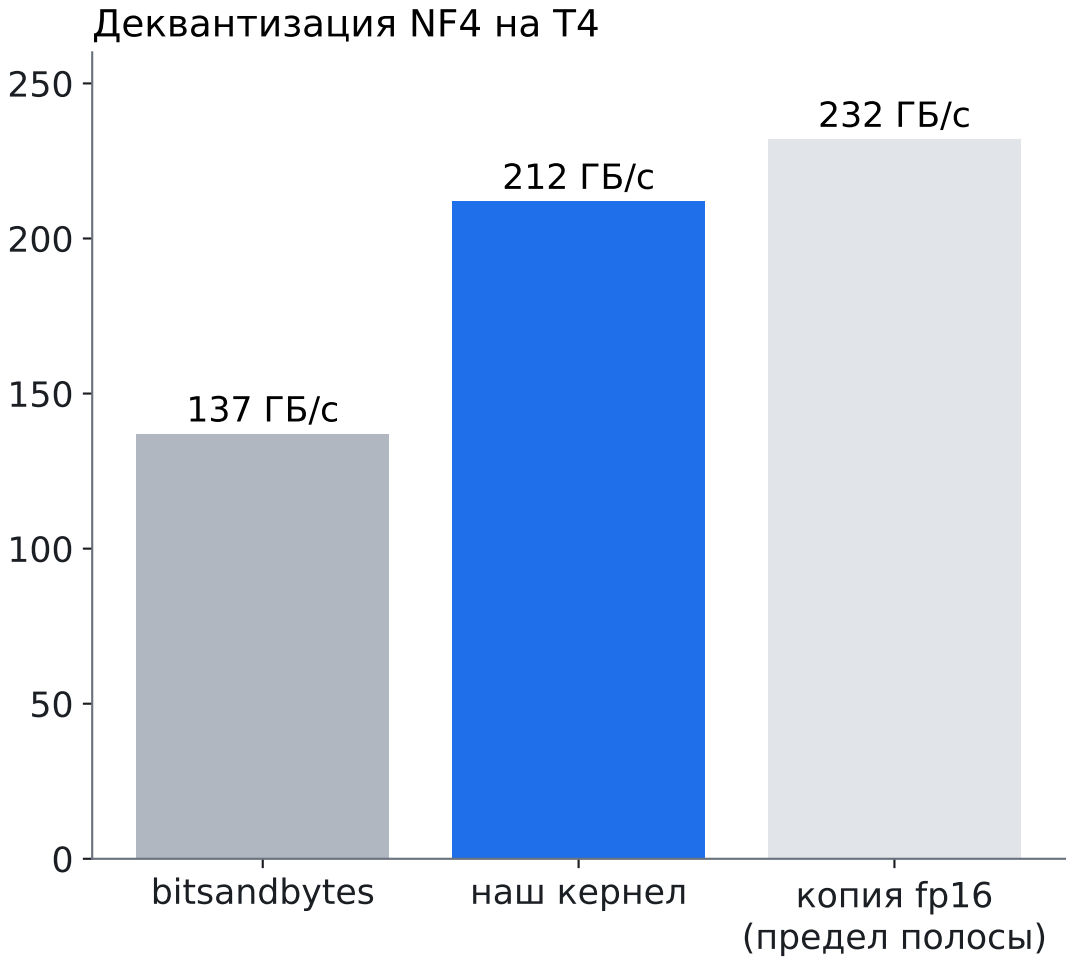
Те же округления до fp16 в тех же местах, fp32 для residual, норм, RoPE, softmax и loss -> кривая loss в 0,01-0,05% от базы.

Наши кернелы



- Ни одного tl.dot; пример шаблона удалён.
- rms_fwd за один проход: residual add, RMSNorm в fp32, каст в fp16 и входы обеих LoRA с dropout; маска бит в бит как у nn.Dropout базы (тот же поток Philox от генератора CUDA), не хранится, в backward считается заново.
- Шаг на GPU: GEMM cuBLAS ~69% работы; GPU занят 82-88%: щели между мелкими запусками ~20 мс/шаг, граница шага тренера HF 8-15 мс.
- Идеи (код наш): Liger-Kernel, Unsloth, Cut Cross-Entropy, QLoRA, online softmax, ZeRO-Offload - ссылки в README.

Кернел крупным планом: деквантизация NF4 под Turing



- Узкое место было не в полосе памяти, а в загрузках таблицы на каждый элемент (~100 мс на шаг).
- Тайл из блоков, уровни NF4 из констант, absmax двойной квантизации побитно как в bitsandbytes: ~100 -> 36 мс на шаг.
- Один запуск на [q|k|v] и один на [gate|up] вместо двух кернелов bnb на вес.
- 23 варианта перепробовали; остановились у предела полосы.
- Speedup всего решения 1,39x -> 1,53x от одного этого кернела.

Голова кусками словаря, вес на CPU

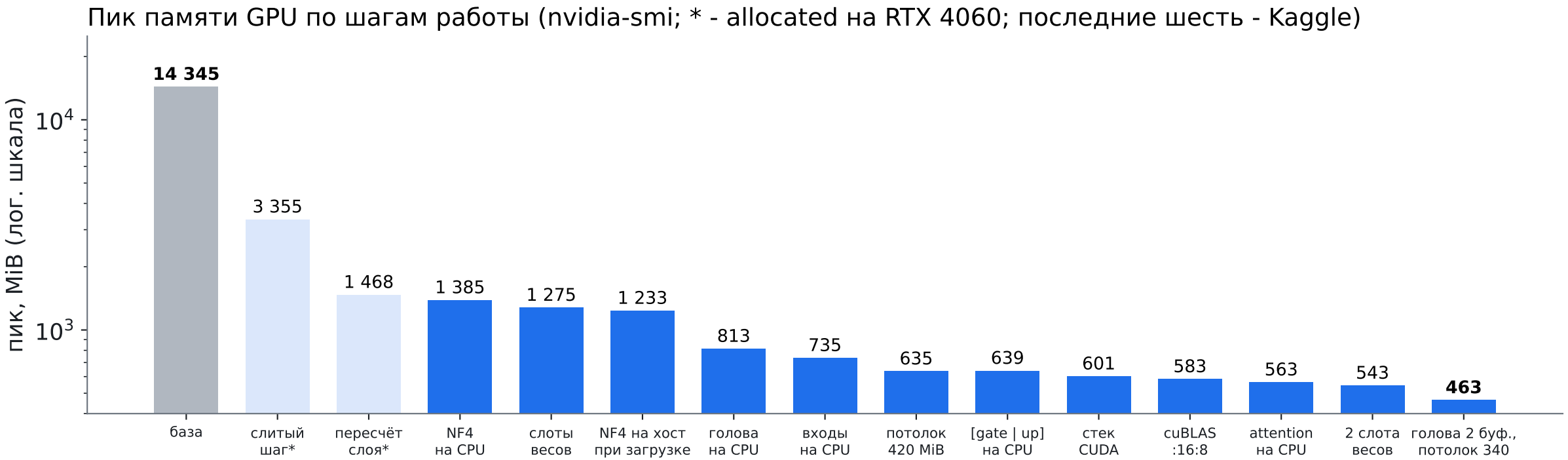
вес lm_head 151 936 x 1 536 (445 MiB) в pinned-памяти CPU: 19 кусков по 8 192 строки



на GPU 3 буфера по 24 MiB: 2 заполняются ещё на последних 3 слоях forward, третий - в начале головы

- На каждый кусок: GEMM lm_head (cuBLAS) -> lm_head_piece: online-состояние строки (максимум, сумма exp, argmax) и накопление градиента с пересчётом при росте максимума.
- В конце lm_head_finish: loss, энтропия, accuracy и градиент. Логиты на весь словарь не существуют ни в какой момент.
- Строки промпта (~37% токенов) получают только энтропию и argmax: без backward-GEMM через матрицу 151 936 x 1 536.
- Loss, энтропия, accuracy и loss строк для rewind - в том же проходе, значения как у базы (проверка 5). Честно: против головы на GPU loss совпадает до ~1e-7, не побитно.

Откуда 31x по памяти



На GPU нет ни одного замороженного веса целиком и ни одного отложенного входа слоя: нормы, LoRA, 2 слота весов слоя, буферы головы, кольца и активации текущего слоя.

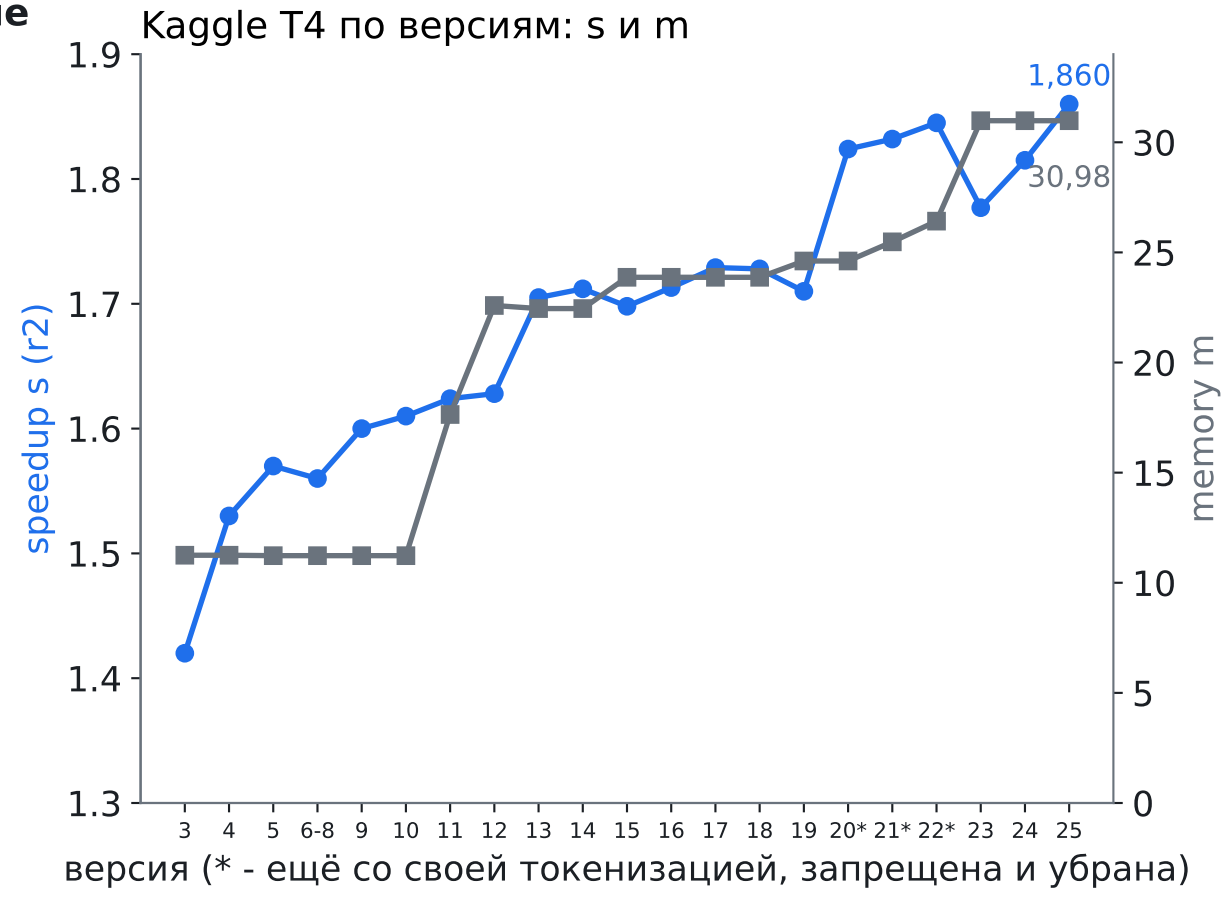
Шаг сам прогнозирует свой пик и тратит запас под потолком 340 MiB на хранение вместо пересчёта. 463 MiB на Kaggle; на 25 срезах Brev 461-481 MiB, 499 на батчах 4 x 512. Ниже ~461 пик держит фрагментация аллокатора (его настройки харнесса менять нельзя).

Результаты против базы

	База	Решение	Отношение
soup train, с	79,0	42,5	**1,860x**
цикл 64 шагов, с	57,5	24,7	**2,32x**
медиана шага, с	0,916	0,350	**2,62x**
пик GPU, MiB	14 345	463	**30,98x**
eval loss	2,04968	2,04962	**-0,003%**
проверки			**10 / 10**

Kaggle T4, --repeats 2 (база, решение, решение, база), 838827f (версия 25: быстрый выход, поздний прогрев, NF4 в pinned); медиана - среднее двух запусков.

Срезы, похожие на скрытые (Brev T4): 32 прогона на 25 срезах, все 10/10, пик 461-481 MiB; на батчах 4 x 512 база падает с OOM, решение - 499 MiB.



Что не сработало

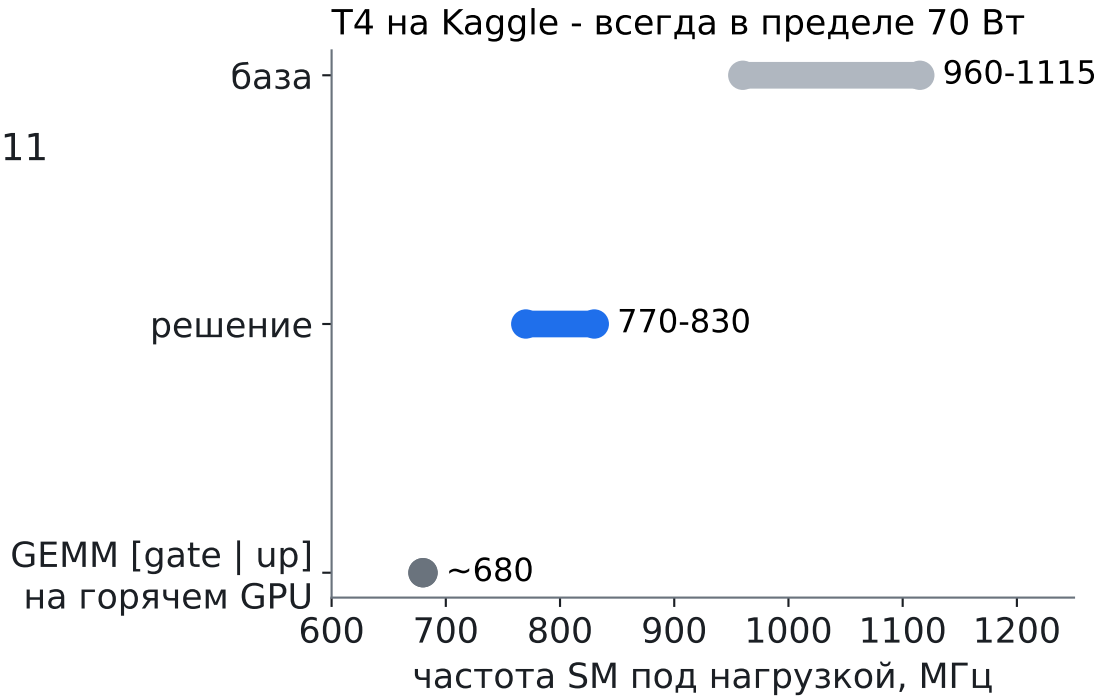
- Свой GEMM на Triton: tl.dot на sm_75 без тензорных ядер - ~1 TFLOPS против 16-34 у cuBLAS.
- int8-GEMM (разрешён): в forward [gate | up] - первый loss до 1,29% при допуске 1%; только в backward - точность как у fp16 и -58 MiB, но на Kaggle медленнее на 1,6-2,3 с (квантизация упирается в медленный CPU).
- Загрузка модели в фоновом потоке: 54,8 с против 54,3 - Python-код загрузки делит GIL.
- CUDA Graphs: длина батча меняется каждый шаг, корзины стоят 3-7% лишних GEMM.
- Голова с резидентными на GPU кусками (4 и 8 по 24 MiB): цикл не быстрее, пик выше.
- Потолок шага ниже 340 MiB: allocated падает, а пик nvidia-smi - нет (фрагментация); при 300 - экономные шаги.
- Свой fingerprint для datasets.map и своя токенизация без map (-2..-3 с): организаторы запретили - убрали.

Каждую гипотезу меряли A/B в одной сессии T4 и оставляли только подтверждённое. Полный журнал - в README.

Ограничения и дальнейшие шаги

- Заточено под Qwen2 + NF4 + LoRA на q/v в fp16 (T4); остальное - исходный код Soup.
- GEMM ~69% работы GPU и упираются в cuBLAS; вне цикла ~11 с дают патчи запуска (с быстрым выходом и поздним прогревом).
- Выгрузки зависят от PCIe: на Kaggle (~12 ГБ/с) - в пределах разброса; тензоры attention уходят на хост только от 9 ГБ/с.
- Loss не побитно равен варианту с головой на GPU (~1e-7); длинные батчи +9% цикла.

Дальше: GEMM на тензорных ядрах T4 через mma.sync внутри Triton; bf16-путь для Ampere; LoRA на всех проекциях.



Внутри запуска решения ~860 -> ~750 МГц,
от запуска к запуску цикл 26,75 -> 28,22 с;
r2 одним ноутбуком - от 1,598x до 1,860x.

github.com/BAITC-Hacks/hack-fe30cb28-opus-5-5-ultracode kaggle.com/code/magurin/soup-kernel-challenge-opus-ultracode